MODULE 9

Build and Dependency Management with Maven

Automate builds, manage dependencies, and standardize the Java development lifecycle for enterprise projects.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 9 OVERVIEW

Enterprise applications depend on hundreds of third-party libraries — Maven manages dependencies, builds, packaging, and testing.

- A Spring Boot application requires Spring Framework, JUnit, Kafka, and Oracle Drivers — managing these versions and their own dependencies by hand is impossible at enterprise scale.

DURATION & LAB

3 Hours Theory

Maven fundamentals, POM, lifecycle, dependencies.

45-Min Timed Lab

Maven Build and Dependencies.

Business Scenario

A Spring Boot app with 4+ major dependencies.

MAVEN MANAGES
Dependencies, Builds, Packaging, Testing

WITHOUT MAVEN
Manual JARs, version conflicts, broken builds

WITH MAVEN
Consistent, repeatable, automated builds

KEY TAKEAWAY Maven turns dependency chaos into a declarative, repeatable build process every developer and CI server can trust. Build automation reduces manual work, improves repeatability, and supports faster feedback.

LEARNING OBJECTIVES

By the end of this module, you will automate builds and manage dependencies like an enterprise Java team.

1. Understand why Maven is used in Java enterprise projects.
2. Create and structure a Maven project using standard conventions.
3. Understand the Project Object Model (POM) and pom.xml.
4. Declare and manage project dependencies.
5. Use Maven goals and lifecycle phases effectively.
6. Build, test, package, and run a project with Maven.
7. Work with Maven plugins and repositories.
8. Understand dependency scopes and transitive dependencies.
9. Analyze and resolve conflicts using the dependency tree.
10. Apply Maven best practices in enterprise build pipelines.

WHAT YOU WILL GAIN

Standardized Builds

Dependency Control

Repeatable Releases

Enterprise Readiness

KEY TAKEAWAY Maven standardizes project structure, automates builds, and simplifies dependency management for scalable, professional Java applications.

MAVEN ARCHITECTURE

Apache Maven is a build automation and dependency management tool for Java projects. Maven is built with a clean, modular architecture that separates core functionality from user-facing features.

Maven Core

- Core build lifecycle and dependency engine.

Plugin Architecture

- Plugins provide goals like compile, test, package.

Project Object Model (POM)

- Unit of work: info, config, dependencies, plugins.

Repository System

- Stores and retrieves artifacts (Local, Remote, Central).

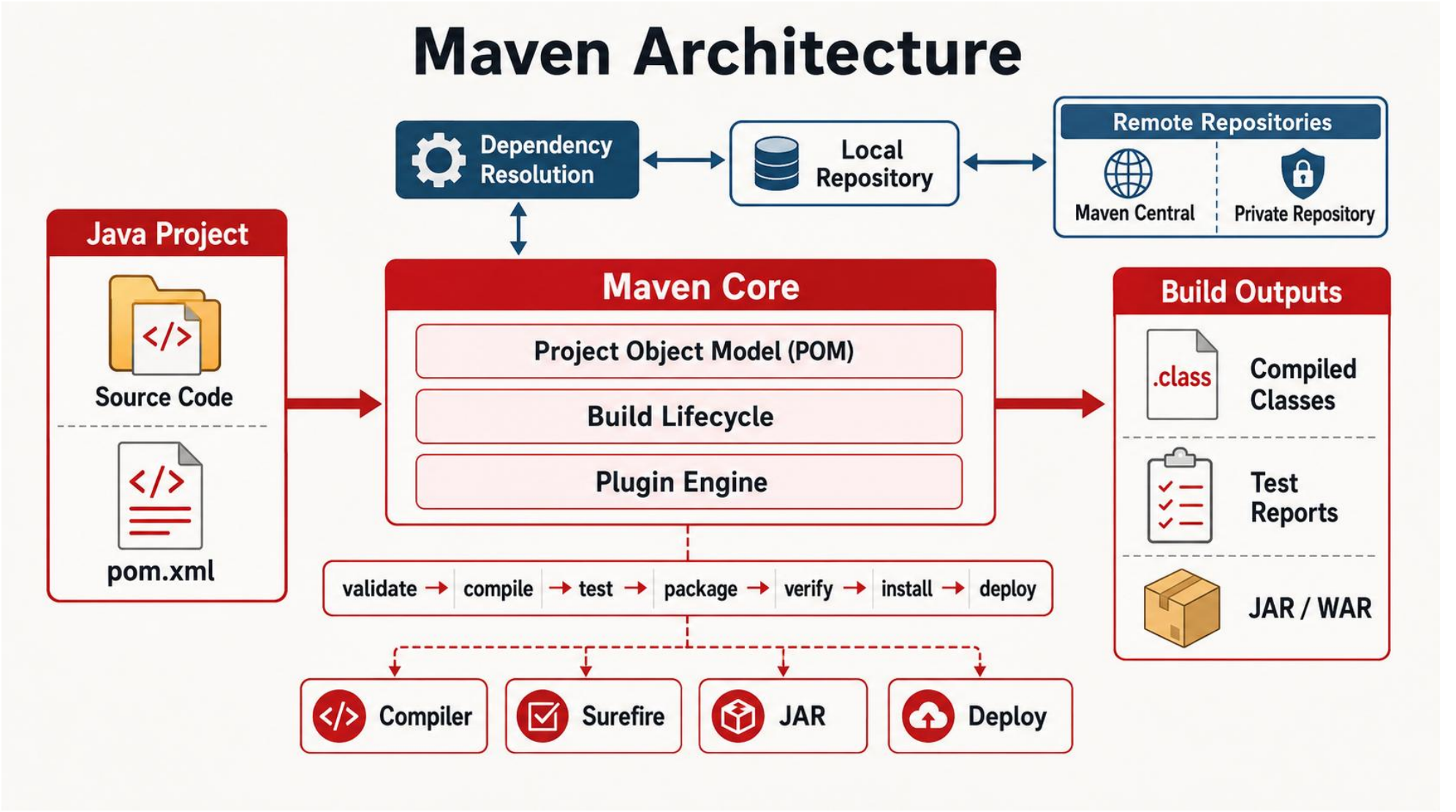
Settings

- Configures repositories, profiles, proxies, credentials.

Convention over Configuration

- Sensible defaults reduce repeated config.

KEY TAKEAWAY Each Maven component has one clear responsibility and communicates through well-defined interfaces — build automation done right.



REPOSITORY RESOLUTION FLOW

Maven always checks the local repository first, then falls back to remote repositories — caching every artifact it downloads for next time.

REPOSITORY TYPES

Local Repository

On the developer's machine (~/.m2); acts as a cache for offline builds.

Central Repository

Maven Central — default public repository, operated by Sonatype.

Third-Party Repos

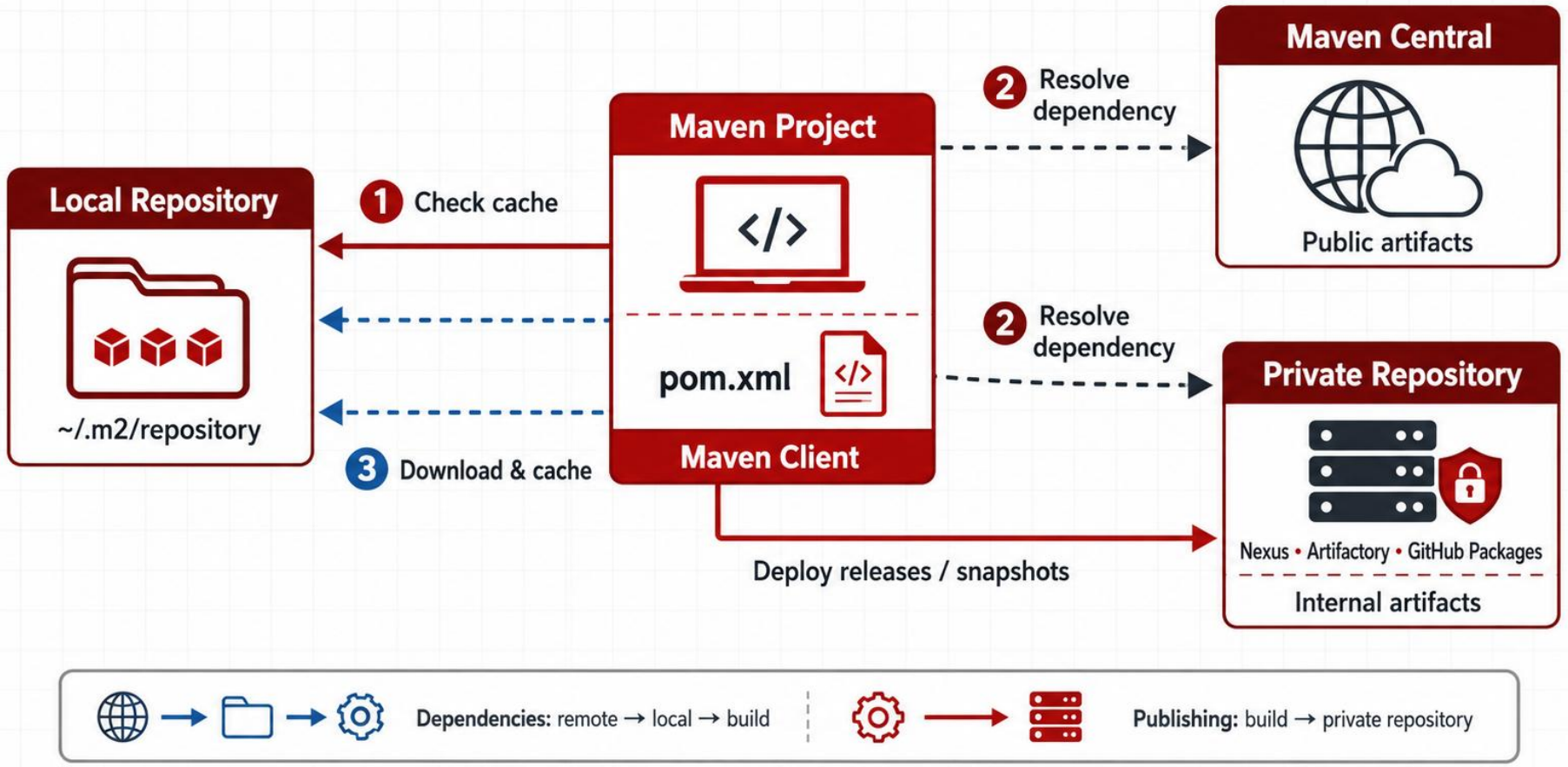
JCenter (archived), Google, JBoss, Spring, etc.

Corporate / Private

Internal artifacts via Nexus, Artifactory, Azure Artifacts.

KEY TAKEAWAY The local-first lookup and cache keep builds fast and mostly offline-capable after the first download.

Maven Repository Ecosystem



LOCAL, CENTRAL & CORPORATE REPOSITORIES (1 OF 2)

Maven Central is operated by Sonatype and is Maven's default public repository, reached through the standard repository endpoint — compare it with the local cache and corporate options.

REPOSITORY TYPES COMPARED

- Local (~/.m2/repository) — private cache on the developer's machine; nothing is shared with the team.
- Central — Maven's default public repository, operated by Sonatype; read-only and open to anyone.
- Corporate / Private — internal repository (Nexus, Artifactory, Azure Artifacts) mirrors Central and hosts internal artifacts.
- All three sit in the same resolution chain: local cache is checked first, then corporate/Central, and results are cached locally.

ARTIFACT COORDINATES (EXAMPLE ONLY)

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
</dependency>
```

KEY TAKEAWAY Local caches speed up rebuilds, Central (operated by Sonatype) supplies the public ecosystem, and corporate repositories keep private artifacts under your organization's control.

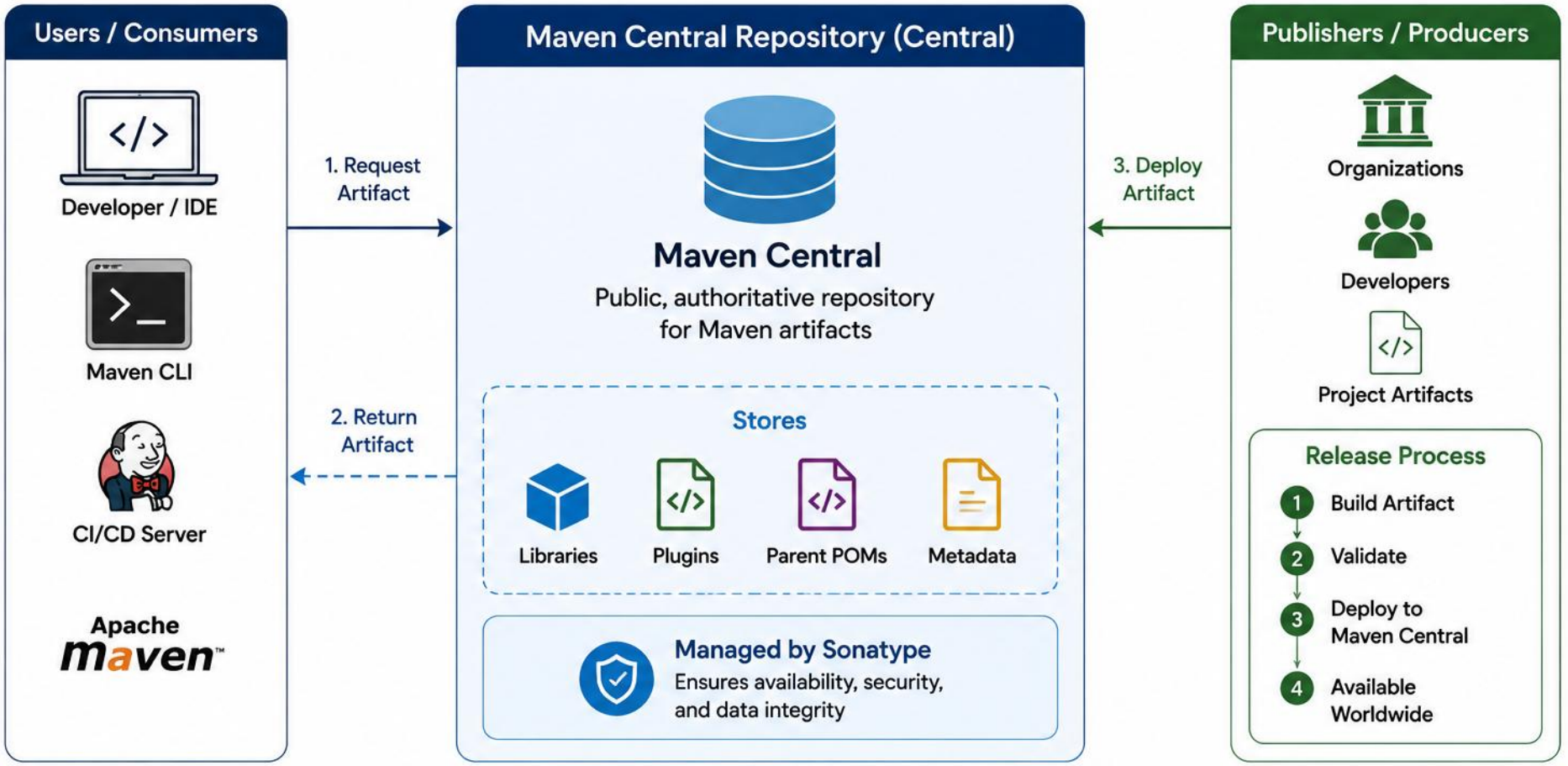
LOCAL, CENTRAL & CORPORATE REPOSITORIES (2 OF 2)

Maven Central is operated by Sonatype and is Maven's default public repository, reached through the standard repository endpoint — compare it with the local cache and corporate options.

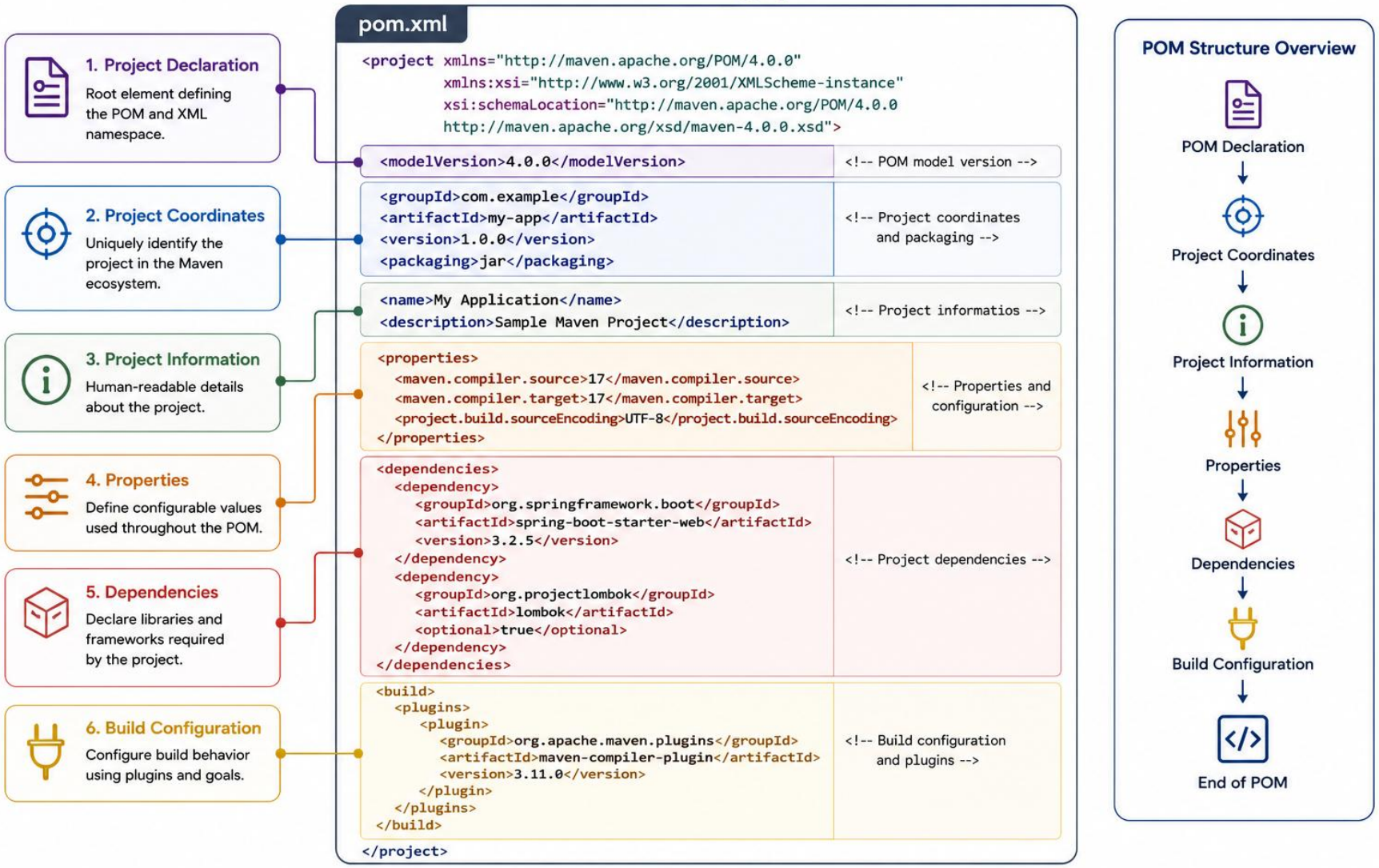
QUICK FACT	VALUE
`Hosted by`	Sonatype
`URL`	https://repo.maven.apache.org/maven2/
`Role`	Default public repository
`Artifacts`	1M+ (and growing)

KEY TAKEAWAY Maven Central is reached through one well-known endpoint, maintained by Sonatype, and hosts 1M+ artifacts — treat its coordinates as stable infrastructure.

Maven Central Repository



Understanding pom.xml



GAV = groupId : artifactId : version the three coordinates that together uniquely identify every Maven project and artifact in a repository.

PROJECT COORDINATES

The unique identity of a Maven project in a repository — groupId, artifactId, and version, together.

```
com.example : my-app : 1.0.0
```

groupId

- Organization identity via reverse domain notation.

artifactId

- Specific project/artifact name (letters, numbers, hyphens).

version

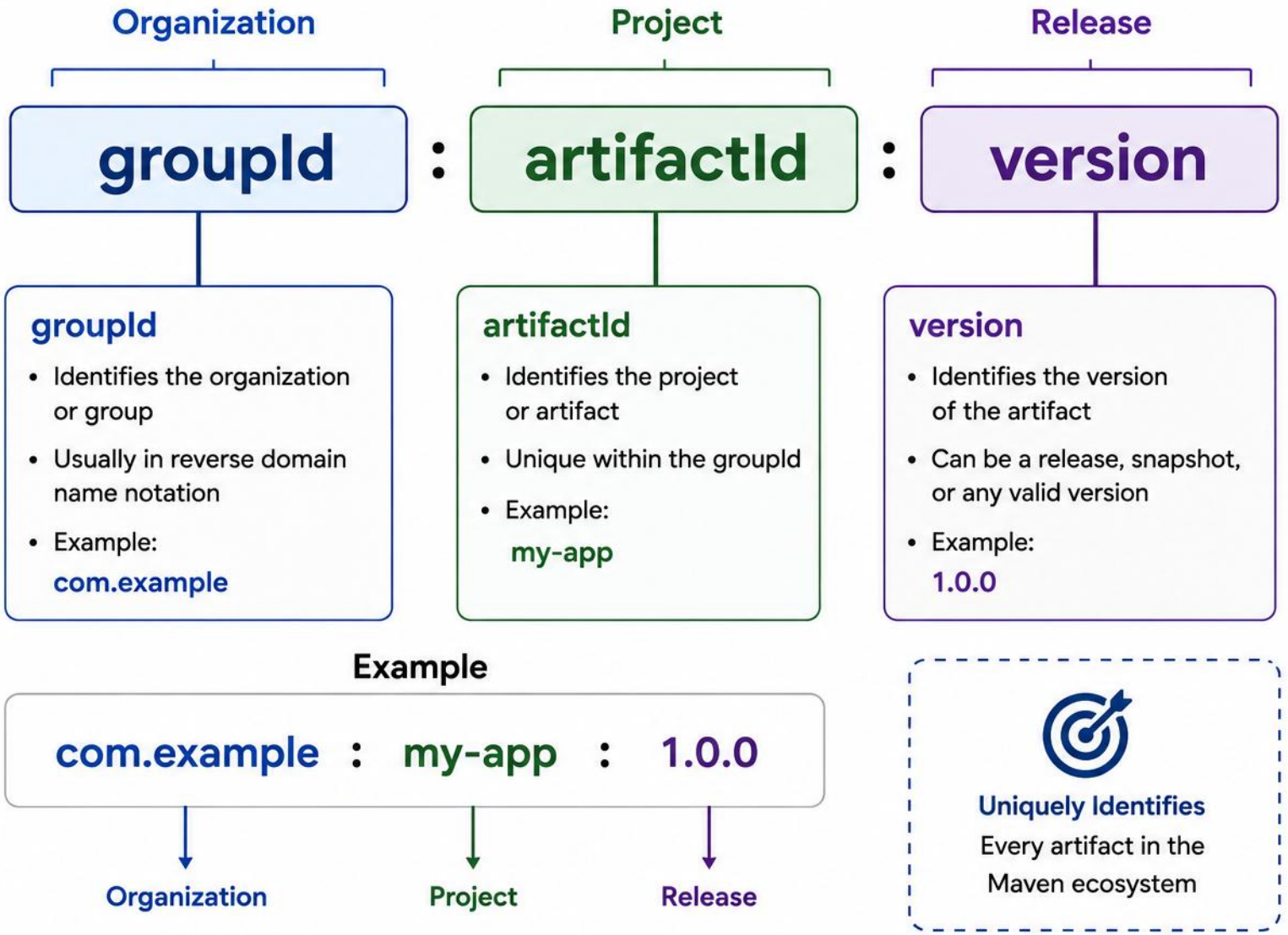
- The release version — follow semantic versioning.

HOW MAVEN USES COORDINATES

1. **Needs Artifact** — Maven reads the coordinates.
2. **Searches Repos** — Local first, then remote.
3. **Downloads** — Artifact is cached locally.
4. **Used in Build** — Right artifact, right version, every time.

KEY TAKEAWAY Project coordinates are the unique address of your project in the Maven world.

Project Coordinates



TRY IT YOURSELF — EXERCISE 1: READ POM COORDINATES

Reinforces: the groupId : artifactId : version coordinates you just saw.

STEP	WHAT TO DO
Goal	Create pom-coordinates-notes.md and state the exact GAV plus packaging for a real POM fragment
File	pom-coordinates-notes.md (in examples/module-09-exercises/)
Coordinates	groupId com.northstar, artifactId customer-service, version 0.1.0-SNAPSHOT, packaging jar
GAV	com.northstar:customer-service:0.1.0-SNAPSHOT
Key concept	-SNAPSHOT = still under active development; may change without a new version
Reference	exercises/exercise-01-pom-coordinates.md

```
$ cat pom.xml
<groupId>com.northstar</groupId>
<artifactId>customer-service</artifactId>
<version>0.1.0-SNAPSHOT</version>  // still under active development
<packaging>jar</packaging>  // GAV: com.northstar:customer-service:0.1.0-SNAPSHOT
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-pom-coordinates.md.

DEPENDENCIES

Dependencies declare the libraries your project needs — Maven automatically downloads and manages them.

EXAMPLE: DECLARING A DEPENDENCY

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.1.6</version>
  </dependency>
</dependencies>
```

KEY POINTS

- Declared inside the <dependencies> section of pom.xml.
- Each dependency is identified by its GAV coordinates (the version shown is illustrative — use the version required by your project or BOM).
- Maven resolves and downloads dependencies automatically.
- Dependencies can bring in their own transitive dependencies.

KEY TAKEAWAY Maven Central hosts the artifact once declared; your job is to describe what you need, not how to fetch it.

DEPENDENCY ATTRIBUTES

Every dependency you declare carries these four attributes — get them right and Maven does the rest.

DEPENDENCY ATTRIBUTES:

Direct vs. Transitive

Direct: you declare it.
Transitive: pulled in by a dependency you declared.

Version

Pins the exact release Maven resolves and downloads.

Scope

Controls which classpath the dependency is available on.

Exclusion

Removes one unwanted transitive dependency using <exclusions>.

KEY TAKEAWAY Every dependency carries these four attributes — get the scope and version right, and use exclusions sparingly to avoid pulling in unwanted transitive code.

EXCLUDING TRANSITIVE DEPENDENCIES

Use <exclusions> to drop one unwanted transitive dependency without touching the dependency that pulls it in.

EXAMPLE: EXCLUDING A TRANSITIVE DEPENDENCY

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>6.1.6</version>
  <exclusions>
    <exclusion>
      <groupId>commons-logging</groupId>
      <artifactId>commons-logging</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

KEY POINTS

- <exclusions> wraps one or more <exclusion> blocks inside a single <dependency>.
- Each <exclusion> needs only groupId and artifactId — no version required.
- Here spring-context keeps its own logging while commons-logging is dropped.
- Exclusions apply per direct dependency — the same library can still arrive another way.

KEY TAKEAWAY Exclude only what actually conflicts or bloats the classpath — excluding too aggressively can silently remove code a dependency needs.

Managing Dependencies

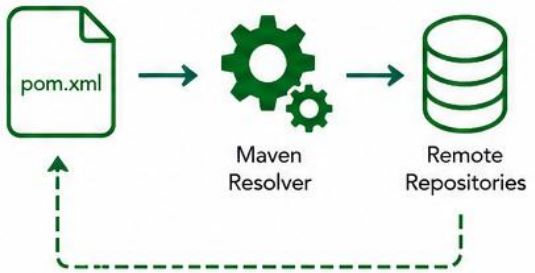
1. Declare Dependencies (pom.xml)

Maven uses GAV coordinates to identify a dependency.

```
<dependencies>
   <dependency>
      <groupId>org.apache.commons</groupId>
      <artifactId>commons-lang3</artifactId>
      <version>3.14.0</version>
      <scope>compile</scope>
   </dependency>
</dependencies>
```

groupId	Identifies the organization or project that published the artifact.
artifiactId	Identifies the name of the artifact.
version	Specifies the version of the artifact.
scope	Defines the context in which the dependency is needed.

2. Dependency Resolution



Downloads and caches dependencies

3. Where Dependencies Come From

Maven Central
Default public repository

Remote Repositories
Company or third-party repositories

Local Repository
Cached on developer's machine (~/.m2/repository)

4. Dependency Scope

compile
Defaullt scope.
Required for compilation and runtime.

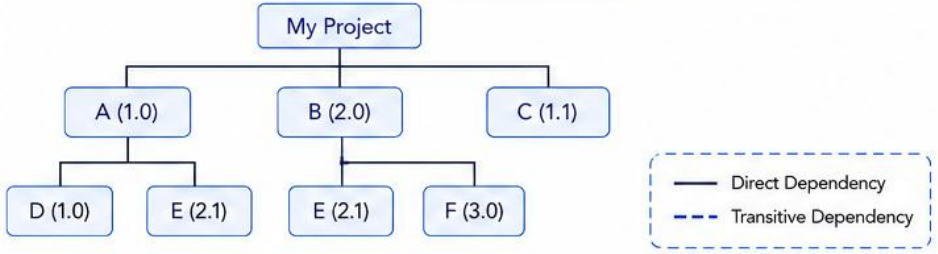
provided
Provided by JDK or container at runtime.
Not packaged.

runtime
Required at runtime but not for compilation.

test
Only for test compilation and execution.

system
Provided as a system path dependency.
Not recommended.

5. Dependency Tree



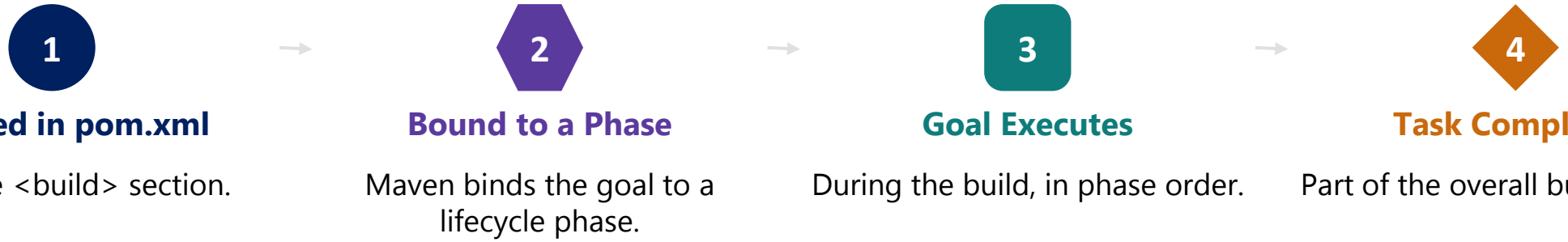
6. Conflict Resolution

Nearest definition wins (or use dependencyManagement)

PLUGINS

Plugins extend Maven's build lifecycle — they provide goals that compile code, run tests, package, generate reports, and more.

HOW PLUGINS WORK



PLUGIN	PURPOSE	COMMON GOAL
`maven-compiler-plugin`	Compiles Java source code	compile, testCompile
`maven-surefire-plugin`	Runs unit tests	test
`maven-jar-plugin`	Packages the project into a JAR	jar
`maven-resources-plugin`	Manages resources	resources:resources

KEY TAKEAWAY Plugins are the power behind Maven's lifecycle — they automate tasks and make builds reliable and consistent across environments.

Plugins

Maven plugins are reusable components that perform specific build tasks at different phases of the build lifecycle.

What are Plugins?

Extend Maven's capabilities to compile code, run tests, package, deploy and more.

In pom.xml

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>17</source>
        <target>17</target>
      </configuration>
    </plugin>
  </plugins>
</build>
```

How Plugins Work

- Plugin is bound to a phase in the lifecycle
- Goal(s) of the plugin are executed
- Build task is completed successfully

Plugins in Build Lifecycle

validate	compile	test	package	verify	install	deploy
Validate the project	Compile the source code	Run unit tests	Package the artifact	Verify the package	Install to local repository	Deploy to remote repository
maven-enforcer-plugin	maven-compiler-plugin	maven-surefire-plugin	maven-jar-plugin	maven-checkstyle-plugin	maven-install-plugin	maven-deploy-plugin

Common Plugin Examples

Plugin	Purpose
maven-compiler-plugin	Compiles the source code
maven-surefire-plugin	Runs unit tests
maven-jar-plugin	Packages classes into a JAR/WAR
maven-deploy-plugin	Deploys artifacts to remote repository

Benefits

- Extensible and Reusable
- Automates Build Tasks
- Ensures Consistency
- Improves Productivity

MAVEN BUILD PROFILES

Profiles switch build configuration for different environments — dev, test, prod — without editing the POM itself.

PROFILE	ACTIVATION	TYPICAL USE
`dev`	<activeByDefault>true</activeByDefault>	Local laptop defaults (never real secrets)
`test`	mvn -Ptest ...	CI test-stage configuration
`prod`	mvn -Pprod ...	Production build settings
`(any)`	mvn help:active-profiles	Verify what is actually active

DECLARING A PROFILE (pom.xml)

- Each profile lives in a <profile> block inside <profiles>, with its own <id> and <properties> (or dependencies/plugins) that override the base POM.
- Multiple profiles can be active at the same time — keep each one focused and composable, and verify the effective result with mvn help:active-profiles and mvn help:effective-pom.
- Prefer environment variables, external configuration, or a secrets manager for runtime settings and secrets; use Maven profiles for build-specific configuration only.

KEY TAKEAWAY Profiles keep dev/test/prod configuration out of source code — activate the right one intentionally, and confirm it with mvn help:active-profiles.

DECLARING PROFILES IN POM.XML

Each profile lives in its own <profile> block inside <profiles> — one can activate by default, another only when you ask for it.

EXAMPLE: DEV PROFILE (ACTIVE BY DEFAULT)

```
<profile>
  <id>dev</id>
  <activation>
    <activeByDefault>true</activeByDefault>
  </activation>
  <properties>
    <app.env>dev</app.env>
  </properties>
</profile>
```

EXAMPLE: PROD PROFILE (EXPLICIT ACTIVATION)

```
<profile>
  <id>prod</id>
  <properties>
    <app.env>prod</app.env>
  </properties>
</profile>

// activate explicitly
$ mvn -Pprod package
```

KEY TAKEAWAY A profile changes properties like app.env, not what code compiles — dev stays the laptop default, prod is activated on purpose.

TRY IT YOURSELF — EXERCISE 2: ACTIVATE BUILD PROFILES

Reinforces: the dev / test / prod profile table you just saw.

STEP	WHAT TO DO
Goal	Create profiles-notes.md; name the default profile, then activate prod deliberately
File	profiles-notes.md (in examples/module-09-exercises/)
Default	dev is active by default (<activeByDefault>true</activeByDefault>)
Activate prod	mvn -Pprod package — never make prod the laptop default
app.env values	dev profile → app.env=dev; prod profile → app.env=prod
Reference	exercises/exercise-02-profiles.md

```
$ mvn package              // dev active by default
$ mvn -Pprod package       // activate prod explicitly
$ mvn help:active-profiles  // confirm what is actually active
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-profiles.md.

MAVEN BUILD LIFECYCLE OVERVIEW

Maven uses a standard build lifecycle to automate building, testing, packaging, and deploying your application.

THREE LIFECYCLE GROUPS

Default Lifecycle

- Build & deploy the app: validate → deploy.

Clean Lifecycle

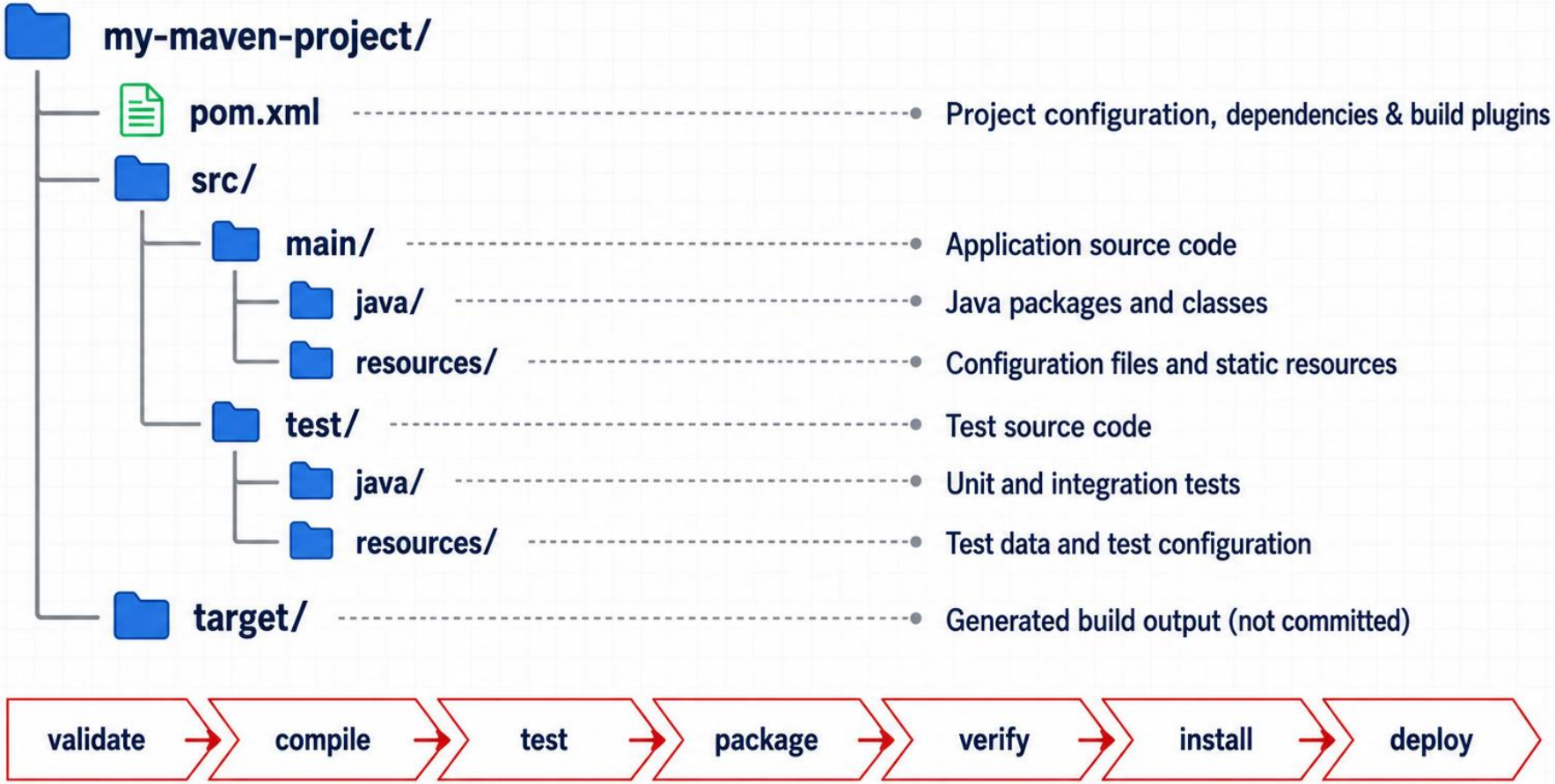
- Cleans artifacts from previous builds: pre-clean → post-clean.

Site Lifecycle

- Creates documentation: pre-site → post-site.

KEY TAKEAWAY Running a phase also runs every preceding phase — 'mvn install' executes validate through install in order.

Standard Maven Project Structure



VALIDATE, COMPILE, AND TEST

The first three phases of the default lifecycle — prepare the project, compile the code, then prove it works.

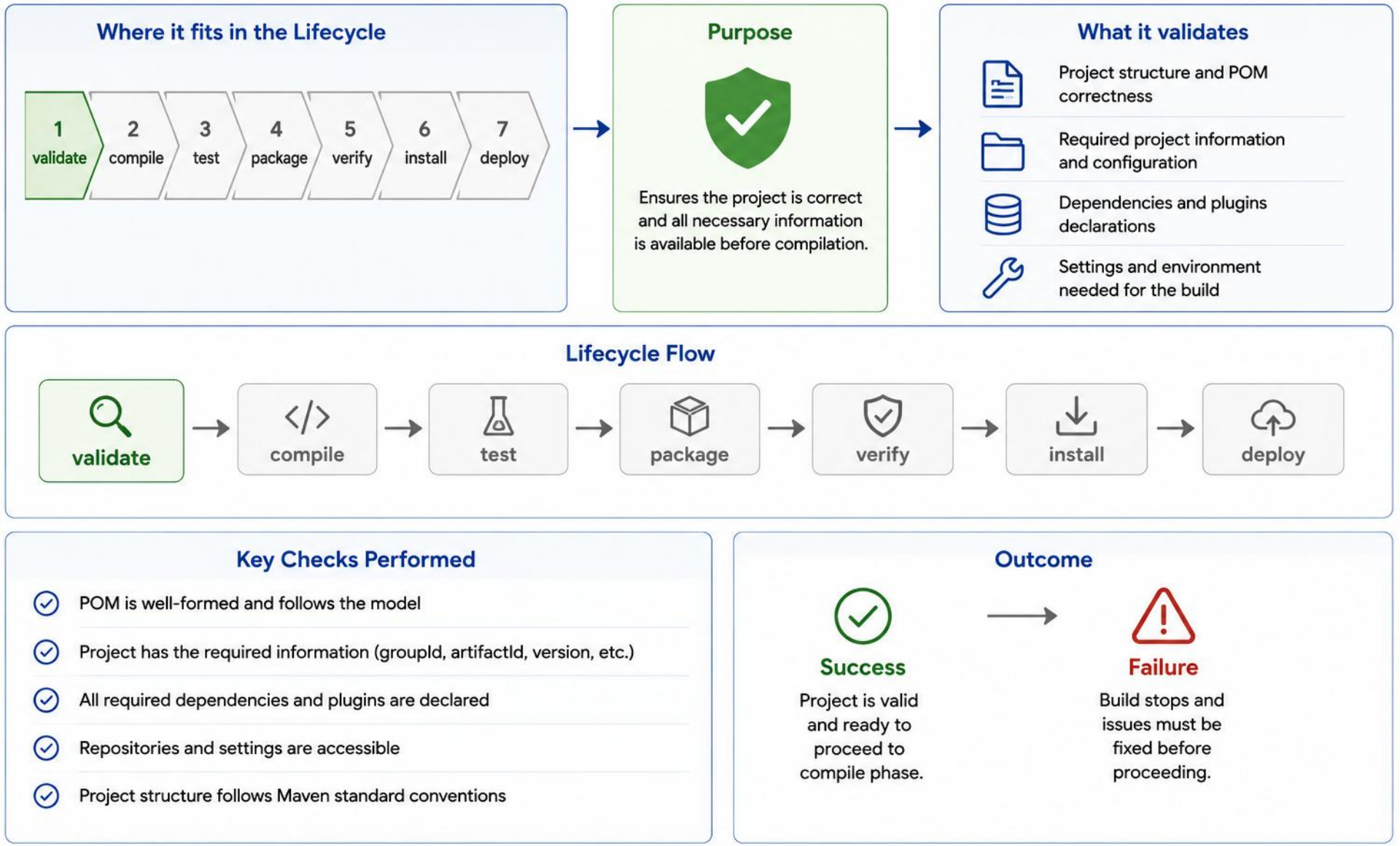
PHASE	COMMAND & WHAT IT DOES	KEY OUTPUT
validate	$ mvn validate — Maven validates the project model automatically at startup; add maven-enforcer-plugin for extra rules.	BUILD SUCCESS
compile	$ mvn compile — compiles src/main/java to target/classes. mvn clean compile always compiles from scratch.	BUILD SUCCESS
test	$ mvn test — Maven Surefire runs tests using the framework declared in the project, such as JUnit or TestNG.	Tests run: 24, Failures: 0

SAMPLE OUTPUT (mvn validate)

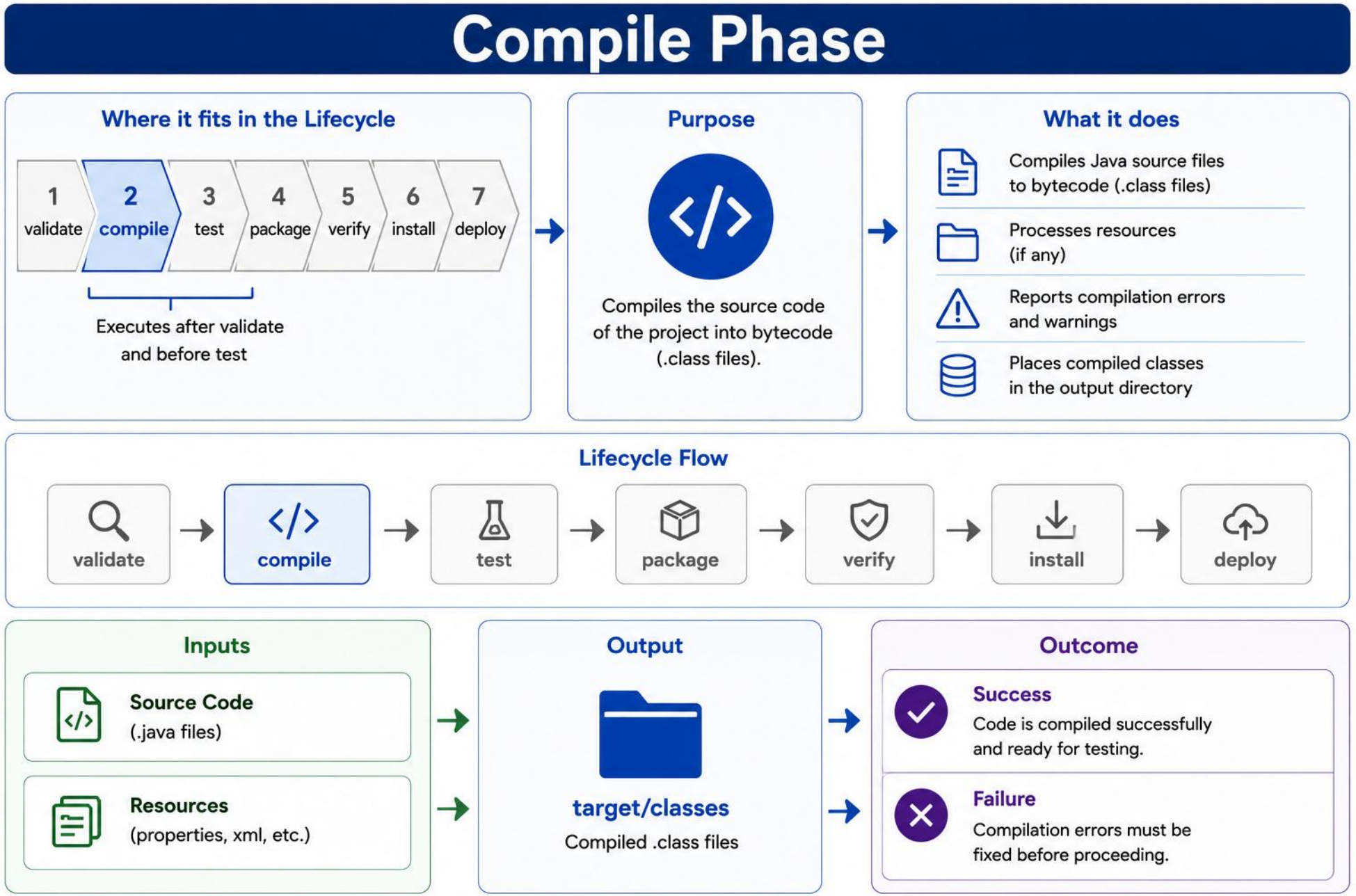
```
[INFO] Scanning for projects...
[INFO] Building banking-app 1.0.0
[INFO] BUILD SUCCESS
```

KEY TAKEAWAY Maven validates the model, compiles from source, then runs tests — each phase must pass before the next one starts.

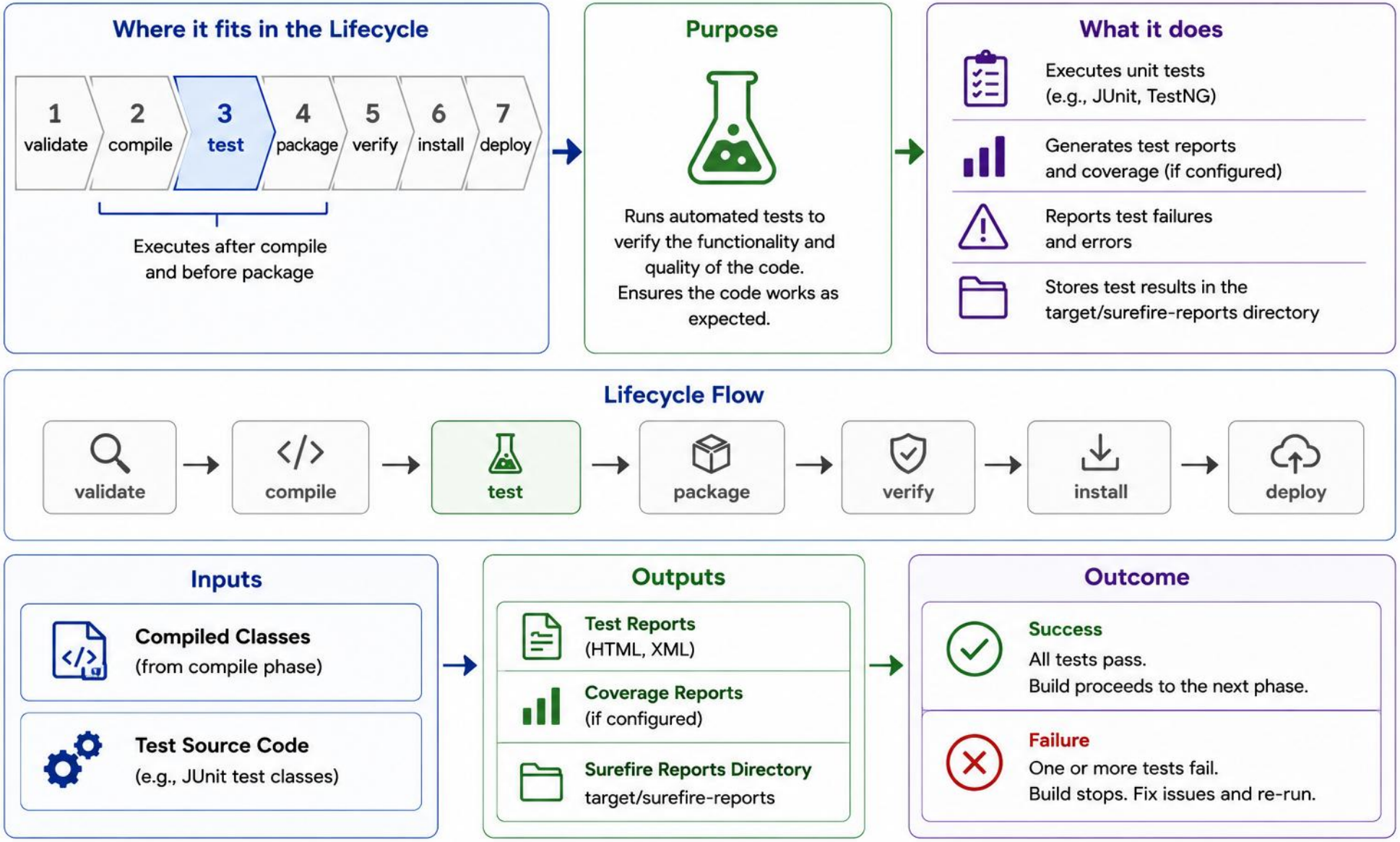
Validate Phase



Compile Phase



Test Phase



PACKAGE AND VERIFY

Package produces a deployable artifact; verify runs whatever quality checks are configured before install or deploy.

PHASE	COMMAND & WHAT IT DOES	KEY OUTPUT
package	$ mvn package — bundles compiled code and resources into a JAR/WAR; test reports go to target/surefire-reports/.	BUILD SUCCESS (jar/war created)
verify	$ mvn verify — runs any verification goals bound in the POM, such as integration tests, coverage checks, style checks, or security scans.	BUILD SUCCESS (if configured)

SAMPLE OUTPUT (mvn package)

```
[INFO] --- maven-resources-plugin:resources ---
[INFO] --- maven-jar-plugin:jar ---
[INFO] Building jar: target/banking-app-1.0.0.jar
[INFO] BUILD SUCCESS
```

KEY TAKEAWAY Package always builds the JAR/WAR; verify only runs the checks you actually configured in the POM — nothing runs automatically beyond that.

Package Phase

Where it fits in the Lifecycle

Executes after test and before verify

Purpose

Packages the compiled code and resources into a distributable format (JAR, WAR, or EAR).

What it does

- Bundles compiled classes and resources
- Creates the artifact (JAR, WAR, or EAR)
- Includes metadata and manifest information
- Places the artifact in the target directory

Lifecycle Flow

Inputs

- **Compiled Classes** (from compile phase)
- **Resources** (properties, xml, etc.)

Outputs

Artifact (JAR / WAR / EAR)

Located in **target/**

Outcome

Success
Artifact is created successfully and build proceeds to verify.

Failure
Packaging errors cause the build to fail and stop.

Verify Phase

Where it fits in the Lifecycle

Executes after package and before install

Purpose

Verifies that the built artifact meets the quality criteria and is valid for installation in the local repository.

What it does

- Performs integration checks and quality validations
- Runs verification goals (e.g., integration tests, quality gates)
- Generates verification reports and metrics
- Ensures the artifact is ready for installation

Lifecycle Flow

validate → compile → test → package → **verify** → install → deploy

Inputs

- **Packaged Artifact** (from package phase)
- **Verification Configuration** (rules, quality gates, settings)

Outputs

- **Verification Reports** (logs, metrics, results)
- **Status** Artifact verification status (pass/fail)

Outcome

- **Success** Artifact is verified and the build proceeds to the install phase.
- **Failure** Verification failed. Build stops and issues must be fixed.

INSTALL VERSUS DEPLOY

The final two phases both publish the artifact — the only difference is where it ends up.

PHASE	DESTINATION	PURPOSE
install	Local ~/.m2/repository	Use the artifact in other local projects
deploy	Remote repository (e.g. Nexus, Artifactory)	Share the artifact with teams and CI builds

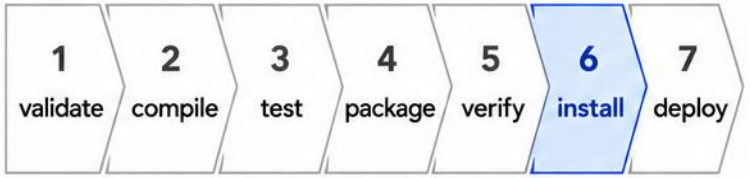
EXAMPLE COMMANDS

```
$ mvn install
[INFO] Installing banking-app-1.0.0.jar to ~/.m2/repository/com/example/banking-app/1.0.0/ — BUILD SUCCESS
$ mvn deploy
[INFO] Uploaded: banking-app-1.0.0.jar to repo.company.com — BUILD SUCCESS
```

KEY TAKEAWAY Rule of thumb: install = local reuse on your machine. deploy = shared with the team and CI/CD.

Install Phase

Where it fits in the Lifecycle



Executes after verify and before deploy

Purpose

Installs the artifact into the local repository for use by other projects on the same machine.

What it does

- Copies the artifact (JAR/WAR/EAR) to the local repository
- Installs POM and metadata in the local repository
- Makes the artifact available for dependency resolution
- Enables other projects to use the artifact locally

Lifecycle Flow

validate → compile → test → package → verify → **install** → deploy

Inputs

- **Verified Artifact** (from verify phase)
- **POM & Metadata** (project information)

Output

- **Local Repository** (~/.m2/repository)

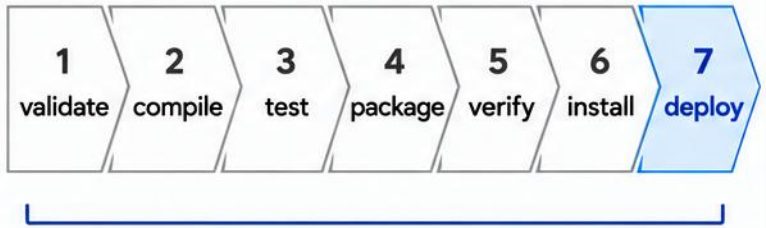
Artifact, POM, and metadata stored for local use

Outcome

- **Success**
  Artifact is installed in the local repository and ready for use.
- **Failure**
  Installation errors cause the build to fail and stop.

Deploy Phase

Where it fits in the Lifecycle



Last phase in the build lifecycle

Purpose

Uploads the built artifact to a remote repository for sharing and release.

What it does

- Publishes the artifact to a remote repository (Nexus / Artifactory / Maven Central)
- Makes the artifact available to other developers and projects
- Deploys with POM and metadata
- Enables dependency sharing across the organization

Lifecycle Flow

Inputs

- **Installed Artifact** (from install phase)
- **POM & Metadata** (project information)

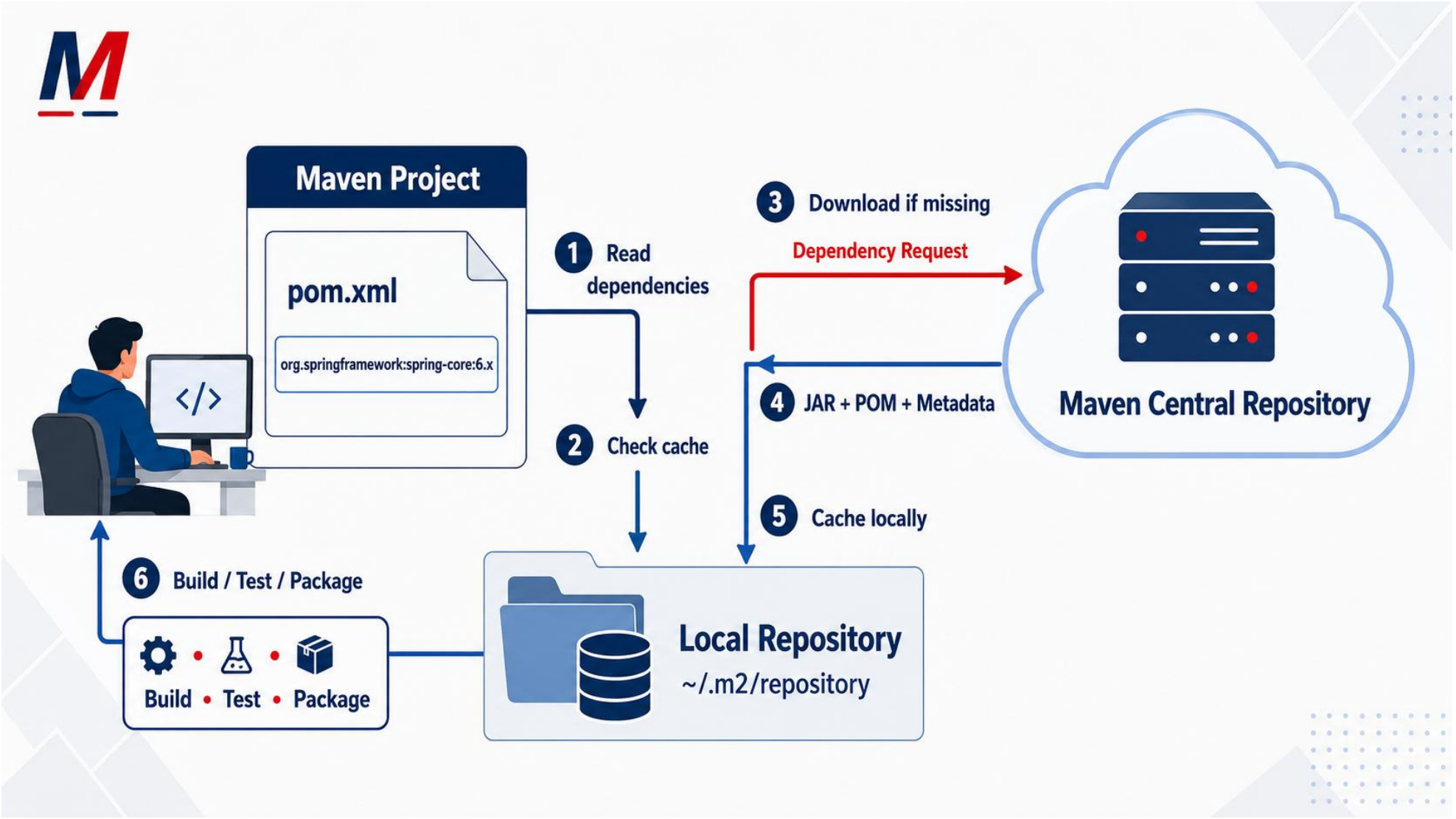
Output

Artifact in Remote Repository (Nexus / Artifactory / Maven Central)

Outcome

Success
Artifact is deployed successfully and available for others to use.

Failure
Deployment errors cause the build to fail and stop.



TRY IT YOURSELF — EXERCISE 3: WALK THE MAVEN LIFECYCLE

Reinforces: the validate → compile → test → package → verify → install → deploy phases you just saw.

STEP	WHAT TO DO
Goal	Create lifecycle-notes.md mapping each phase to its command and what it proves
File	lifecycle-notes.md (in examples/module-09-exercises/)
Order	validate → compile → test → package → verify → install (no deploy in this exercise)
CI habit	mvn -B verify = batch mode; stops after verify, skips install/deploy
Key concept	deploy belongs to release/CI credentialed publishing, not classroom machines
Reference	exercises/exercise-03-lifecycle.md

```
$ mvn validate
$ mvn compile
$ mvn test
$ mvn package
$ mvn -B verify   // then: mvn install for local reuse only — not deploy
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-lifecycle.md.

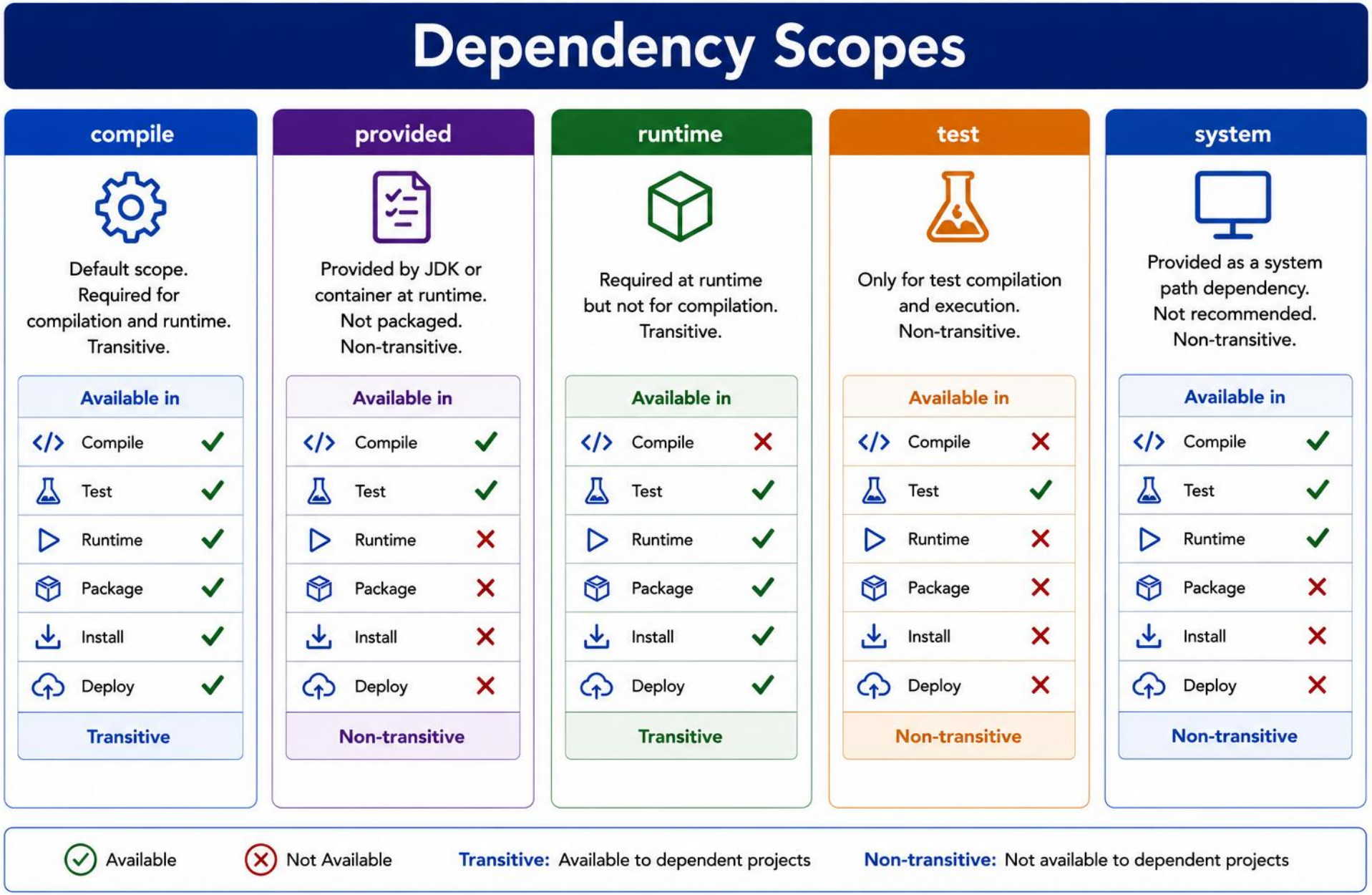
DEPENDENCY SCOPES

Scopes define the purpose and visibility of a dependency across the compile, test, runtime, and packaged classpaths.

SCOPE	DESCRIPTION	COMPILE / TEST / RUNTIME / PKG
`compile`	Default scope — available on every classpath.	Yes / Yes / Yes / Yes
`provided`	Supplied by the JDK/container at runtime (e.g. Servlet API); not packaged.	Yes / Yes / No / No
`runtime`	Needed at runtime, not at compile time. A JDBC driver fits here only when your code depends on the standard JDBC API and loads the driver at runtime.	No / Yes / Yes / Yes
`test`	Only available for test compilation and execution (e.g. JUnit).	No / Yes / No / No
`system`	Local JAR with an explicit path — use rarely; not packaged.	Yes / Yes / No / No
`import`	Used only in <dependencyManagement> to import a BOM.	BOM import only

KEY TAKEAWAY Scopes control which classpath (compile/test/runtime) a dependency lands on and whether it is packaged — get this wrong and you risk ClassNotFoundException or a bloated artifact.

Dependency Scopes



DEPENDENCY SCOPES IN PRACTICE

The same <dependency> block, four ways — <scope> decides which classpath each library lands on.

COMPILE & PROVIDED SCOPES

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
</dependency>  // compile: the default scope
<dependency>
  <groupId>jakarta.servlet</groupId>
  <artifactId>servlet-api</artifactId>
  <scope>provided</scope>
</dependency>
```

RUNTIME & TEST SCOPES

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <scope>test</scope>
</dependency>
```

KEY TAKEAWAY Compile is the default and travels everywhere; provided and test stop at compile time; runtime is added only when the app executes.

TRY IT YOURSELF — EXERCISE 4: CHOOSE DEPENDENCY SCOPES

Reinforces: the compile / test / runtime / provided scope table you just saw.

STEP	WHAT TO DO
Goal	Create dependency-scopes-notes.md; assign the correct scope to 4 real dependency needs
File	dependency-scopes-notes.md (in examples/module-09-exercises/)
Assign	JUnit Jupiter (tests only) → test; Spring Context (production code) → compile (default)
Assign more	JDBC driver (no compile-time import) → runtime; server-supplied API → provided
Common mistake	Leaving JUnit on default compile scope pollutes runtime classpath, misleads CI
Reference	exercises/exercise-04-dependency-scopes.md

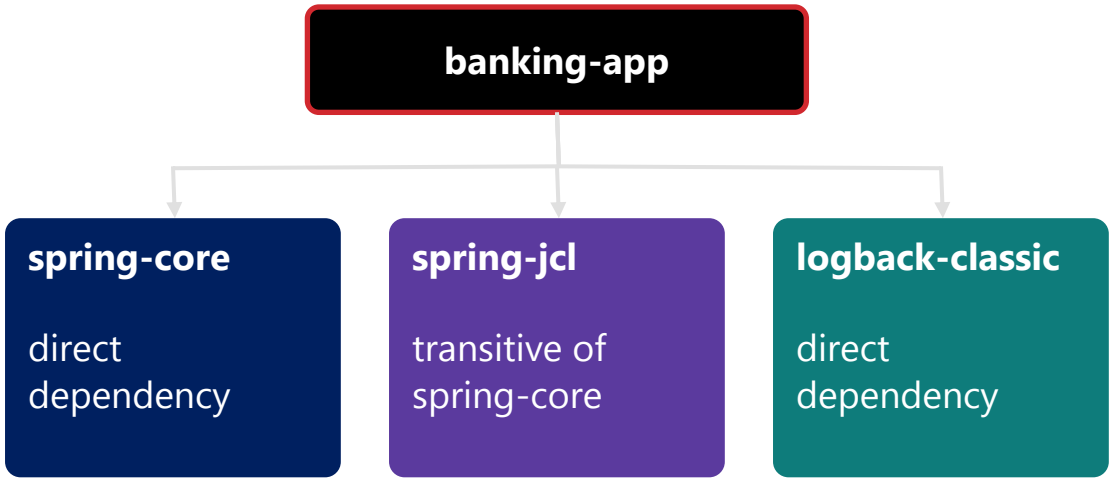
```
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter</artifactId>
  <version>5.11.4</version>
  <scope>test</scope>  // not the default compile scope
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-dependency-scopes.md.

TRANSITIVE DEPENDENCIES

Dependencies of your dependencies — Maven downloads and includes them on the classpath automatically.

EXAMPLE: DEPENDENCY RELATIONSHIP



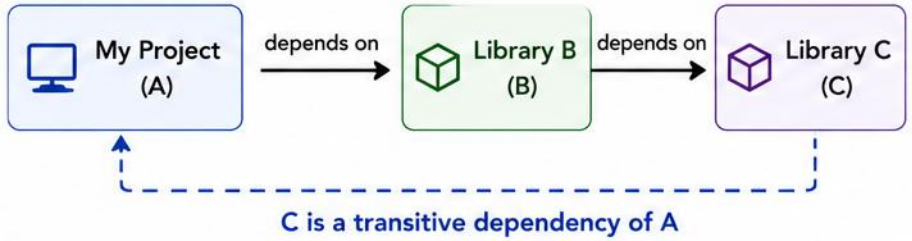
KEY POINTS

- Resolved automatically and added to the classpath.
- Maven handles version conflicts using the 'nearest definition wins' rule.
- You usually don't need to declare transitive dependencies explicitly.
- Declare only what your project directly needs.

KEY TAKEAWAY Transitive dependencies reduce boilerplate — understand what is included, but declare only what you need.

Transitive Dependencies

Dependency Chain



My Project (A) depends on Library B,
Library B depends on Library C,
so Library C is a transitive dependency of My Project.

POM Representation

```
<dependencies>
  <dependency>
    <groupId>com.example</groupId>
    <artifactId>library-b</artifactId>
    <version>1.0.0</version>
  </dependency>
</dependencies>
```

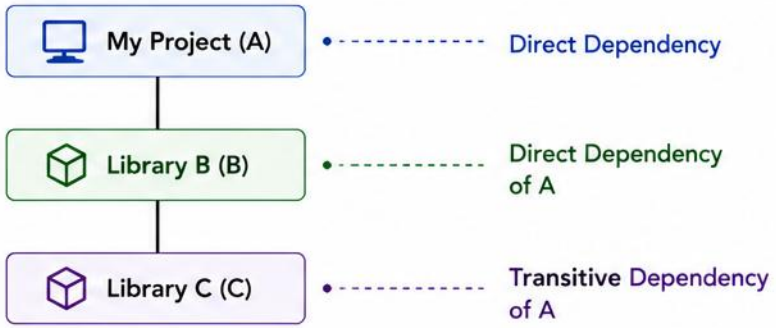
A declares dependency on B

B declares dependency on C (transitive)

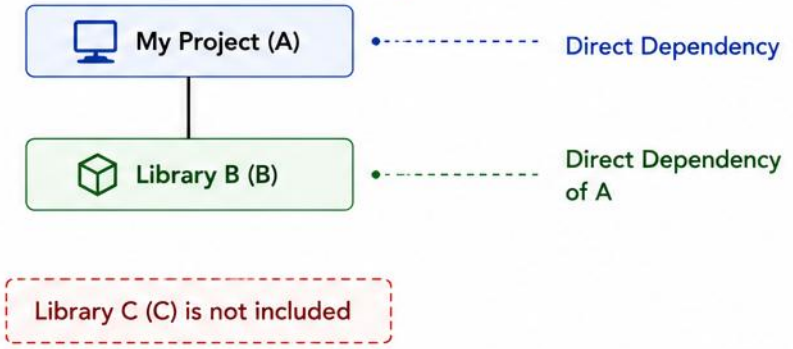
A does not declare C, but it is included automatically through B.

Dependency Tree

With Transitive Dependencies (Default)



Without Transitive Dependencies



A = My Project B = Direct Dependency C = Transitive Dependency

READING THE DEPENDENCY TREE

Every tree line uses the same symbols — learn them once to spot direct, transitive, and excluded dependencies.

EXAMPLE: DEPENDENCY TREE OUTPUT (versions shown are illustrative)

```
$ mvn dependency:tree
[INFO] com.example:banking-app:jar:1.0.0
[INFO] +- org.springframework:spring-core:jar:6.1.7:compile
[INFO] |  \- org.springframework:spring-jcl:jar:6.1.7:compile
[INFO] \- junit:junit:jar:4.13.2:test
```

READING THE SYMBOLS

- +- / \- mark a direct or last child; | shows a sibling.
- (omitted) = excluded due to a conflict or duplicate.
- Each line ends with its scope (compile, test, runtime...).

KEY TAKEAWAY Symbols tell you the shape of the tree — +- / \- mark a direct or last child, | marks a sibling, and (omitted) flags an excluded conflict.

RESOLVING DEPENDENCY CONFLICTS

Maven resolves version conflicts using the nearest-wins rule — override it explicitly when the default pick is wrong.

RESOLVING CONFLICTS (NEAREST WINS)

- Maven picks the version nearest to your POM; at equal depth, the first declared version wins.
- Override via a direct dependency, dependencyManagement, a BOM, or an exclusion.
- Run mvn dependency:tree (or -Dincludes=...) regularly to catch conflicts.

-Dverbose note: mvn dependency:tree -Dverbose shows every version considered, but support varies across Maven Dependency Plugin versions. Plain mvn dependency:tree (with -Dincludes=groupId:artifactId) is the safer, primary command.

KEY TAKEAWAY Symbols tell you the shape of the tree; nearest-wins (with same-depth ties going to the first declaration) tells you which version survives.

PINNING VERSIONS WITH DEPENDENCYMANAGEMENT

<dependencyManagement> fixes the version Maven resolves for a library, even when a transitive dependency asks for something older.

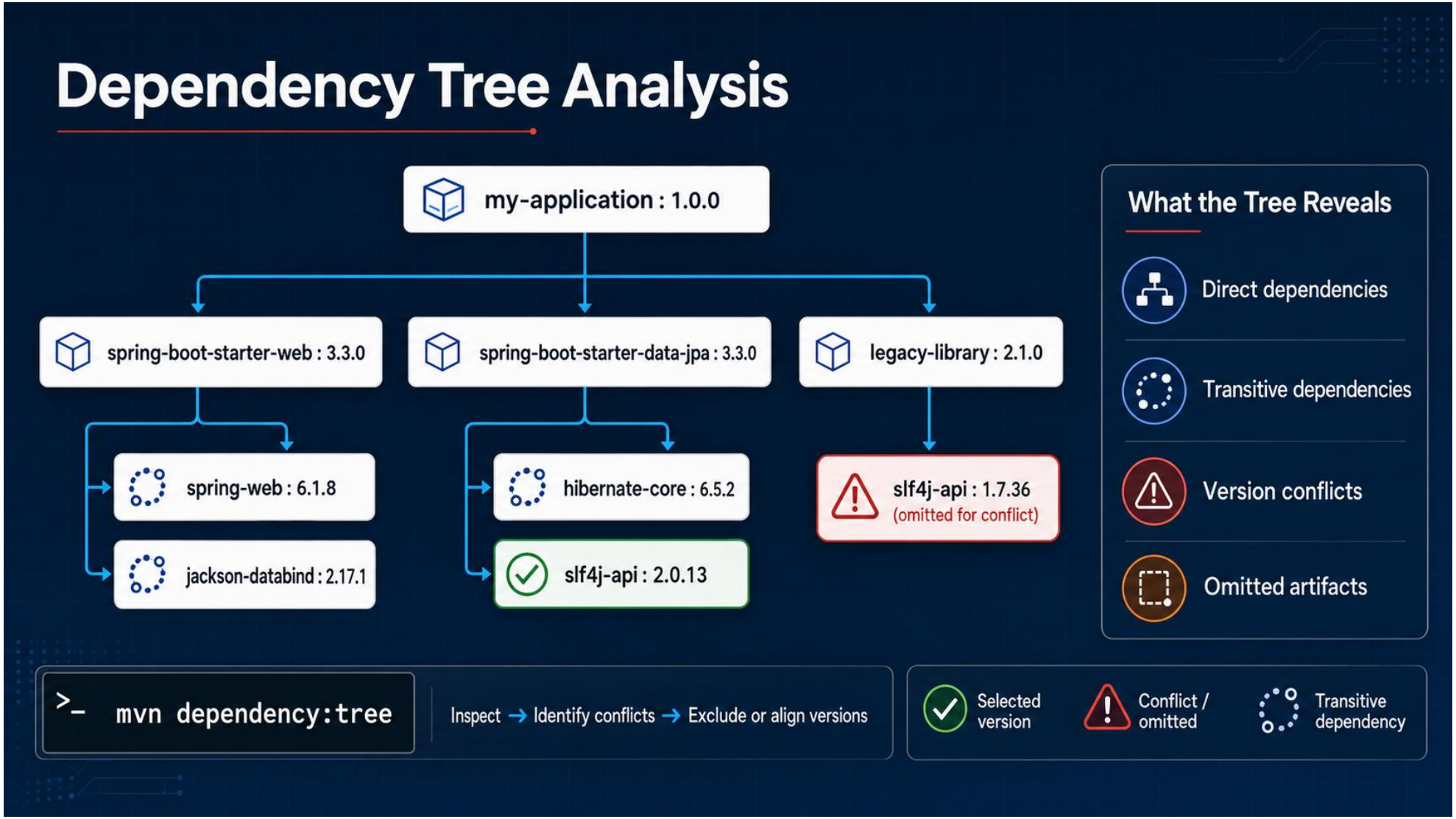
EXAMPLE: PINNING A TRANSITIVE VERSION

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.slf4j</groupId>
      <artifactId>slf4j-api</artifactId>
      <version>2.0.13</version>
    </dependency>
  </dependencies>
</dependencyManagement>
```

KEY POINTS

- dependencyManagement declares a version without adding the dependency to the build.
- Every module or transitive path requesting slf4j-api now resolves to 2.0.13.
- This overrides Maven's normal nearest-wins conflict resolution on purpose.
- Combine with a BOM import (scope=import) to manage many versions from one place.

KEY TAKEAWAY dependencyManagement pins the version once — every dependency and transitive path that needs that artifact inherits it automatically.



TRY IT YOURSELF — EXERCISE 5: READ A DEPENDENCY TREE

Reinforces: the +- / \- symbols and nearest-wins rule you just saw.

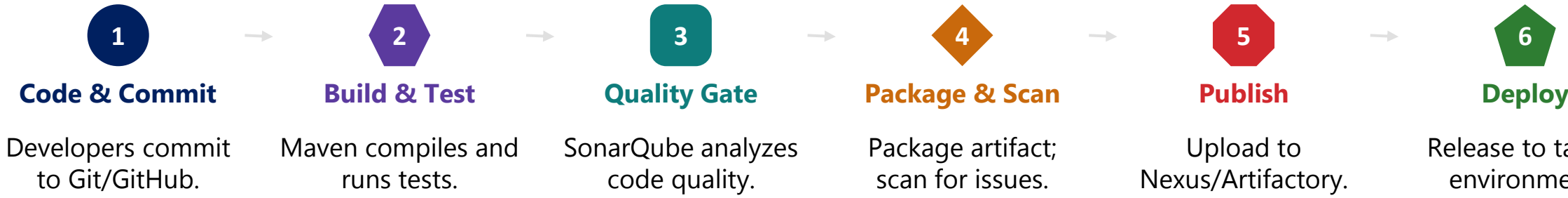
STEP	WHAT TO DO
Goal	Create dependency-tree-notes.md; classify direct vs. transitive, then run the command
File	dependency-tree-notes.md (in examples/module-09-exercises/)
Classify	junit-jupiter = direct (declared); junit-jupiter-params = transitive (via Jupiter)
Command	mvn -q dependency:tree — run from mini-maven/ after Exercise 6
CI habit	-B = batch mode; prefer mvn -B verify over casual install on every push
Reference	exercises/exercise-05-dependency-tree.md

```
$ cd mini-maven
$ mvn -q dependency:tree
com.northstar:build-demo:jar:0.1.0-SNAPSHOT
+- org.junit.jupiter:junit-jupiter:jar:5.11.4:test
   \- org.junit.jupiter:junit-jupiter-params:jar:5.11.4:test  // transitive
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-dependency-tree.md.

ENTERPRISE BUILD PIPELINES

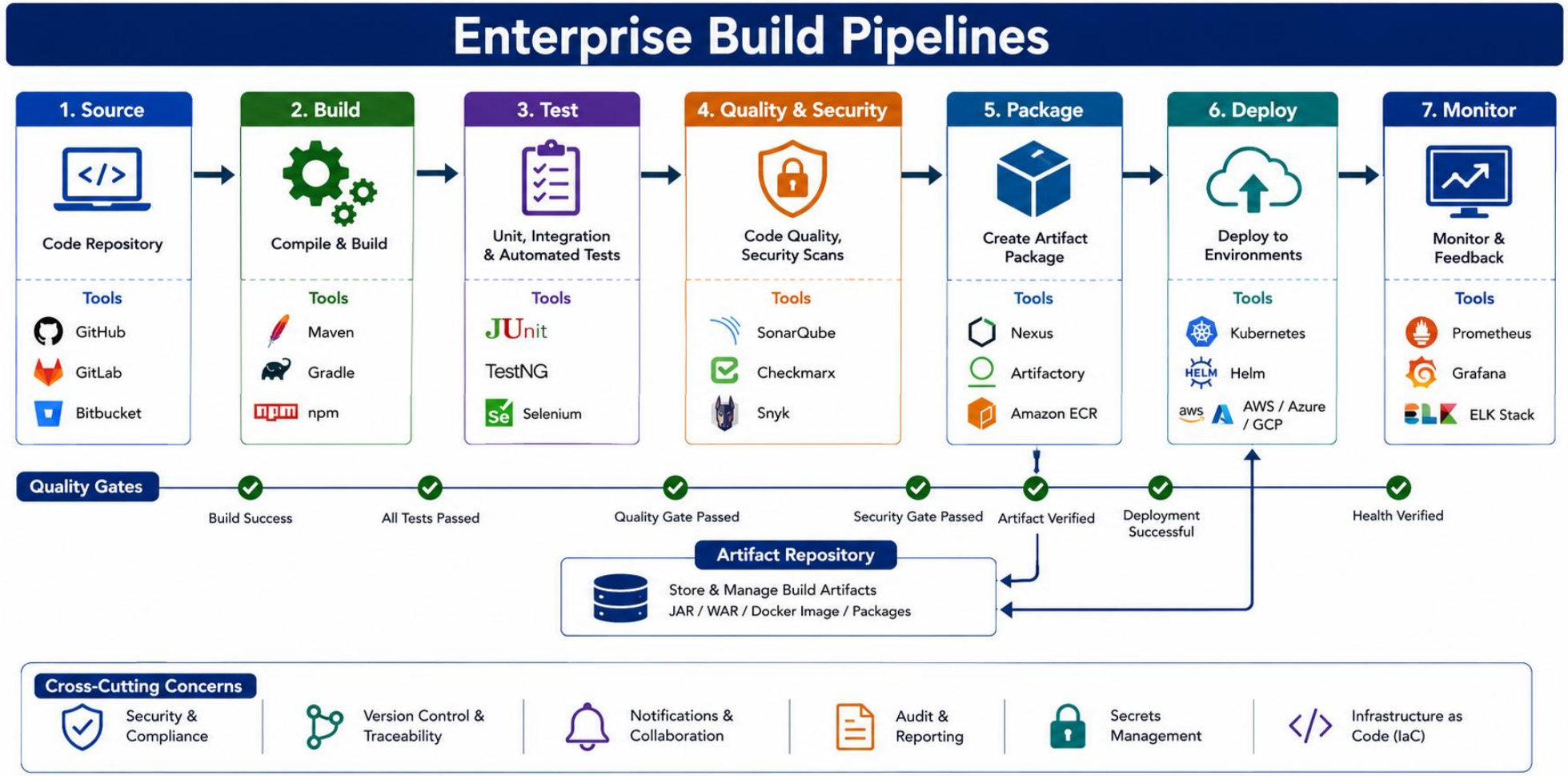
Maven enables consistent, automated, and repeatable builds across environments — quality, security, and faster delivery.



PIPELINE BENEFITS

- **Faster Feedback**
- **Higher Quality & Security**
- **Reduced Manual Errors**
- **Auditability & Traceability**

KEY TAKEAWAY A strong Maven build pipeline is the foundation of reliable, secure, and scalable enterprise delivery.



MAVEN BEST PRACTICES

Follow these proven practices to build reliable, maintainable, and secure Maven projects.

Use a Parent POM

- Centralize shared config and versions.

Keep Dependencies Clean

- Declare only what you use; avoid dynamic version ranges.

Make Builds Reproducible

- Lock plugin/dependency versions; use BOMs and <dependencyManagement>.

Secure Your Supply Chain

- Use trusted repos; scan for CVEs; never store credentials in pom.xml.

Use Semantic Versioning

- Follow MAJOR.MINOR.PATCH releases.

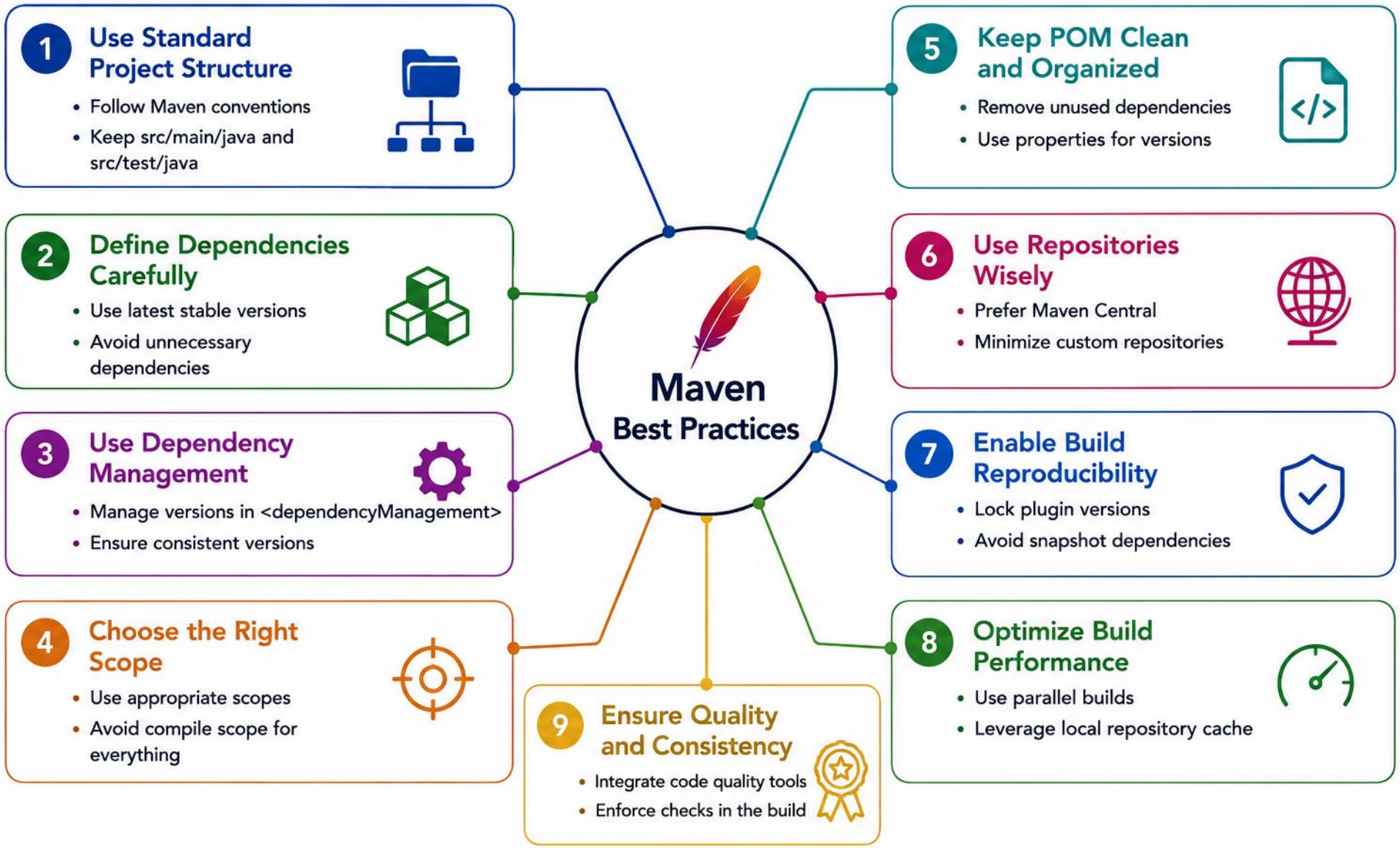
Automate Quality Checks

- Run quality gates on every build.

USEFUL COMMANDS

- mvn clean install — build and install the project. mvn dependency:tree — show the dependency tree. mvn help:effective-pom — view the resolved POM.
- mvn versions:display-dependency-updates — check for dependency updates. ./mvnw clean verify — use the Maven Wrapper so every environment builds with the same Maven version; commit pom.xml and the wrapper files to version control.

KEY TAKEAWAY A well-structured Maven project is easy to build, easy to maintain, and ready for enterprise scale.



PARENT POM AND BOM IMPORTS

A parent POM centralizes shared versions; importing a BOM lets child modules skip <version> entirely.

PARENT POM (dependencyManagement + BOM import)

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-dependencies</artifactId>
      <version>3.3.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

CHILD MODULE POM (inherits + uses the BOM)

```
<parent>
  <groupId>com.northstar</groupId>
  <artifactId>customer-platform</artifactId>
  <version>1.0.0</version>
</parent>
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
</dependencies>
```

KEY TAKEAWAY Children inherit shared dependencyManagement from the parent — declare groupId and artifactId only, and the BOM supplies a consistent version.

TRY IT YOURSELF — EXERCISE 6: FILL A MINI POM (STARTER)

Reinforces: coordinates, scopes, lifecycle phases, and plugins — all four, together, before Lab 9.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — fill blanks so mini-maven/ compiles, tests, packages a JAR
File	mini-maven/pom.xml + src/main + src/test (in examples/module-09-exercises/mini-maven/)
Fill in	7 blanks: groupId, artifactId, version, packaging, JUnit scope, release, mainClass
Run	mvn -q test, then mvn -q package, then java -jar target/build-demo.jar
Expected	Prints "BuildDemo ready for Lab 9"; Surefire reports one passing test
Reference	exercises/exercise-06-mini-pom.md

```
$ cd mini-maven
$ mvn -q test
$ mvn -q package
$ java -jar target/build-demo.jar   // prints: BuildDemo ready for Lab 9
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-mini-pom.md (starter).

LAB OVERVIEW – MAVEN BUILD AND DEPENDENCIES

Expand the Lab 8 CRM skeleton into a build-managed Maven project with dependencies, scopes, plugins, and profiles.

LAB AT A GLANCE

- Starting point: the Lab 8 CRM skeleton (plain Java, no build tooling).
- Target result: customer-service.jar, built validate through install with scopes and profiles demonstrated.
- High-level flow: expand pom.xml, add dependencies & plugins, define profiles, then verify the build.
- Spring dependencies are included only to practise dependency resolution; Spring implementation begins in later modules.

LAB ENVIRONMENT & PREREQUISITES

ITEM	REQUIREMENT
OS	Windows / macOS / Linux
JDK	21 or later
Maven	3.9 or later
IDE	IntelliJ IDEA (primary) / VS Code

SKILLS YOU WILL PRACTICE

POM & Coordinates

Dependency Mgmt

Tree Analysis

Profiles & CI

KEY TAKEAWAY Estimated time: ~45 min (timed path) or 3-4 hrs (full path). Outcome: a working customer-service.jar built validate through install, with scopes and profiles demonstrated.

LAB TASKS AND DELIVERABLES (1 OF 2)

Complete the tasks below to expand the Lab 8 CRM skeleton into a build-managed Maven project with dependencies, plugins, and profiles.

#	TASK	KEY CONCEPTS	WEIGHT
1	Copy Lab 8 into lab9-crm and confirm baseline compile	Environment, JDK 21, Maven 3.9+	10%
2	Expand pom.xml: coordinates, scopes, dev/test/prod profiles	GAV, scopes, lifecycle, profiles	30%
3	Configure compiler, Surefire, and jar plugins; package the JAR	Plugins, Main-Class, JAR	15%
4	Run controlled failure experiments (bad version, failing test)	Resolution errors, test failures	15%

Tasks 5-7, deliverables, and submission steps continue on the next slide.

KEY TAKEAWAY Tasks 1-4 build the foundation: baseline compile, POM expansion, and a packaged JAR. Tasks 5-7 continue on the next slide.

LAB TASKS AND DELIVERABLES (2 OF 2)

Complete the tasks below to expand the Lab 8 CRM skeleton into a build-managed Maven project with dependencies, plugins, and profiles.

#	TASK	KEY CONCEPTS	WEIGHT
5	Verify the build with mvn test and mvn -B verify	Automated verification, CI	10%
6	Confirm no secrets in POM or profiles	Secrets hygiene, dev profile safety	10%
7	Document lifecycle evidence, dependency tree, and README	lifecycle-evidence.md, dependency-tree.txt	10%

DELIVERABLES

- Completed pom.xml (dependencies, plugins, profiles), PlaceholderTest, docs/lifecycle-evidence.md, docs/dependency-tree.txt, customer-service.jar evidence, controlled-failure evidence, and a README documenting mvn -B verify.

KEY TAKEAWAY Submit via the lab portal: capture evidence under notes/screenshots/lab-9, and confirm 'mvn -B verify' passes before final submission.

MAVEN BUILD CHECKLIST

Run through this checklist before you call a Maven build done.

1. Project builds clean with mvn -B verify.
2. Dependencies use the correct scopes.
3. Plugin versions are pinned, not floating.
4. Dependency tree has no unexpected conflicts.
5. Profiles contain no secrets.
6. Artifact is reproducible build to build.
7. Build is CI-ready (mvn -B verify passes in CI).

Run mvn -B verify locally before every push; let CI confirm it one more time.

KEY TAKEAWAY A build that passes every item here is dependable, secure, and ready to hand to another developer or a CI server.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Maven concepts, dependency management, and build best practices.

1. What is the primary purpose of Maven?

A. Code compilation only
B. Build automation and dependency management
C. Database management
D. Version control

2. What is a transitive dependency?

A. A dependency declared with scope 'test'
B. A dependency pulled in by another dependency
C. A dependency declared in the parent POM
D. A dependency installed manually

3. Which file defines a Maven project?

A. build.xml
B. pom.xml
C. settings.xml
D. Mavenfile

4. Which command shows the dependency tree?

A. mvn show:dependencies
B. mvn dependency:tree
C. mvn tree:dependencies
D. mvn list:tree

KEY TAKEAWAY mvn dependency:tree displays the project's full dependency tree, including transitive dependencies.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. Which is NOT a standard Maven directory?

A. src/main/java
B. src/test/java
C. src/main/resources
D. lib

6. How does Maven resolve version conflicts by default?

A. Uses the first declared version
B. Uses the nearest wins rule
C. Uses the latest version
D. Throws a build error

7. Which scope is the default in Maven?

A. provided
B. compile
C. runtime
D. test

8. Which plugin generates code coverage reports?

A. Maven Compiler Plugin
B. Surefire Plugin
C. JaCoCo Plugin
D. Dependency Plugin

ANSWER KEY

- **1-B 2-B 3-B 4-B 5-D 6-B 7-B 8-C**

KEY TAKEAWAY Mastering Maven build and dependency management helps you deliver reliable, maintainable, scalable Java applications.

"Build automation is essential for modern software development."

MODULE 9 COMPLETE

Build and Dependency Management with Maven

You can now automate builds, manage dependencies with confidence, and apply Maven best practices at enterprise scale.